

Ashwath Raj M| AP25110010712| CSE core A

GITHUB LINK: <https://github.com/Ashwath-Raj/CSE-204-DAA-assignments->

1. Summation of Series

Method 1 — Direct Formula

Problem

Find the sum of the series:

$$1 + X + X^2 + X^3 + \dots + X^n$$

Method

The sum is calculated directly using the geometric series formula.

Output:

```
cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummationMInKandBST/Source-Code/" && g++ 1.1_Series_Method_1.cpp -o 1.1_Series_Method_1 && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignmen
ts-/LAB 2_SummationMInKandBST/Source-Code/"1.1_Series_Method_1

• < and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummationMInKandBST/Source-Code/"1.1_Series_Method_1
=====
SUMMATION OF SERIES USING FORMULA
=====
Series: 1 + X + X^2 + X^3 + ... + X^n
Enter Value of X: 5
Enter Value of n: 2
-----
Sum of the given series = 31
-----
Time Complexity: O(1)
Space Complexity: O(1)

~ /A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code
```

Code:

```
/*
Program Name: Summation of Series - Method 1
Program Description: Takes the inputs {x, n} and calculates the sum of the
series 1 + X + X^2 + X^3 + ... + X^n using the
geometric series formula
Program Creation Time: Aug 13th 03:20
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<cmath>
using namespace std;

double sumGeoSeries(double x, int n);

int main(void) {
```

```
cout << "======" << endl;
cout << "SUMMATION OF SERIES USING FORMULA" << endl;
cout << "======" << endl;
```

```
double x;
int n;
```

```
cout << "Series: 1 + X + X^2 + X^3 + ... + X^n" << endl;
```

```
cout << "Enter Value of X: ";
cin >> x;
```

```
cout << "Enter Value of n: ";
cin >> n;
```

```
if (n < 0) {
    cout << "Invalid value of n!!" << endl;
    return 0;
}
```

```
cout << "-----" << endl;
```

```
double sum = sumGeoSeries(x, n);
```

```
cout << "Sum of the given series = " << sum << endl;
```

```
cout << "-----" << endl;
cout << "Time Complexity: O(1)" << endl;
cout << "Space Complexity: O(1)" << endl;
```

```
return 0;
}
```

```
double sumGeoSeries(double x, int n) {
    if (x == 1)
        return n + 1;
```

```
    return (pow(x, n + 1) - 1) / (x - 1);
}
```

Complexity

Time Complexity: O(1)

Space Complexity: O(1)

Method 2 — Iterative Summation

Problem

Find the sum of the series:

$$1 + X + X^2 + X^3 + \dots + X^n$$

Method

The series is evaluated iteratively by generating each successive term and adding it to the accumulated sum.

Output:

```
• ~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2 SummationMINandBST/Source-Code/" && g++ 1.2_Series_Method_2.cpp -o 1.2_Series_Method_2 && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummationMINandBST/Source-Code/"1.2_Series_Method_2
=====
SUMMATION OF SERIES USING ITERATION
=====
Series: 1 + X + X^2 + X^3 + ... + X^n
Enter Value of X: 12
Enter Value of n: 2
-----
Sum of the given series = 157
-----
Time Complexity: O(n)
Space Complexity: O(1)

• ~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code
```

Code:

```
/*
Program Name: Summation of Series - Method 2
Program Description: Takes the inputs {x, n} and calculates the sum of the
series 1 + X + X^2 + X^3 + ... + X^n using an iterative
approach
Program Creation Time: Aug 13th 03:25
Program Creator: Ashwath Raj | AP25110010712
*/
```

```
#include<iostream>
using namespace std;

double sumGeoSeries(double x, int n);

int main(void) {
    cout << "===== " << endl;
    cout << "SUMMATION OF SERIES USING ITERATION" << endl;
    cout << "===== " << endl;

    double x;
    int n;

    cout << "Series: 1 + X + X^2 + X^3 + ... + X^n" << endl;
```

```
cout << "Enter Value of X: ";  
cin >> x;
```

```
cout << "Enter Value of n: ";  
cin >> n;
```

```
if (n < 0) {  
    cout << "Invalid value of n!!" << endl;  
    return 0;  
}
```

```
cout << "-----" << endl;
```

```
double sum = sumGeoSeries(x, n);
```

```
cout << "Sum of the given series = " << sum << endl;
```

```
cout << "-----" << endl;  
cout << "Time Complexity: O(n)" << endl;  
cout << "Space Complexity: O(1)" << endl;
```

```
return 0;  
}
```

```
double sumGeoSeries(double x, int n) {  
    double sum = 0;  
    double term = 1;
```

```
    for (int i = 0; i <= n; i++) {  
        sum += term;  
        term *= x;  
    }
```

```
    return sum;  
}
```

Complexity

Time Complexity: $O(n)$

Space Complexity: $O(1)$

2. Kth Smallest Element in an Array

Method 1 — Sorting

Problem

Find the Kth smallest element in a given array.

Method

The array is sorted in ascending order. The Kth smallest element is then obtained from position $K - 1$.

Output:

```
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummationMInKandBST/Source-Code/" && g++ 2.1_Kth_Smallest_Sorting.cpp -o 2.1_Kth_Smallest_Sorting && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummationMInKandBST/Source-Code/"2.1_Kth_Smallest_Sorting
=====
FINDING KTH SMALLEST ELEMENT USING SORTING
=====
Enter Size of array to be Created: 5
Enter Element 0 in array: 2
Enter Element 1 in array: 3
Enter Element 2 in array: 7
Enter Element 3 in array: 1
Enter Element 4 in array: 3
-----
Enter value of K: 3
The 3th smallest element is: 3
-----
Time Complexity: O(n log n)
Space Complexity: O(log n)
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code
```

Code:

```
/*
Program Name: Kth Smallest Element Using Sorting
Program Description: Takes the inputs {arr, arr size, k} and finds the Kth
smallest element in the given array by first sorting
the array
Program Creation Time: Aug 13th 04:30
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<algorithm>
using namespace std;

int kthSmallest(int arr[], int n, int k);

int main(void) {
cout << "===== " << endl;
cout << "FINDING KTH SMALLEST ELEMENT USING SORTING" << endl;
cout << "===== " << endl;
```

```

int n = 0;

cout << "Enter Size of array to be Created: ";
cin >> n;

if (n <= 0) {
    cout << "Invalid array size!!" << endl;
    return 0;
}

int arr[n];

for (int i = 0; i < n; i++) {
    cout << "Enter Element " << i << " in array: ";
    cin >> arr[i];
}

cout << "-----" << endl;

int k;

cout << "Enter value of K: ";
cin >> k;

if (k < 1 || k > n) {
    cout << "Invalid value of K!!" << endl;
    return 0;
}

int result = kthSmallest(arr, n, k);

cout << "The " << k << "th smallest element is: "
<< result << endl;

cout << "-----" << endl;
cout << "Time Complexity: O(n log n)" << endl;
cout << "Space Complexity: O(log n)" << endl;

return 0;
}

int kthSmallest(int arr[], int n, int k) {
    sort(arr, arr + n);
    return arr[k - 1];
}

```

Complexity

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$ approximately

Method 2 — Min Heap

Problem

Find the Kth smallest element in a given array.

Method

A Min Heap is constructed using the elements of the array. The minimum element is removed repeatedly until the Kth smallest element is reached.

Output:

```
~ /A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummatiomMinKandBST/Source-Code/" && g++ 2.2_Kth_Smallest_Min_Heap.cpp -o 2.2_Kth_Smallest_Min_Heap && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummatiomMinKandBST/Source-Code/"2.2_Kth_Smallest_Min_Heap
=====
FINDING KTH SMALLEST ELEMENT USING MIN HEAP
=====
Enter Size of array to be Created: 4
Enter Element 0 in array: 1
Enter Element 1 in array: 3
Enter Element 2 in array: 5
Enter Element 3 in array: 2
-----
Enter value of K: 3
The 3th smallest element is: 3
-----
Time Complexity: O(n + k log n)
Space Complexity: O(n)
~ /A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code
```

Code:

```
/*
Program Name: Kth Smallest Element Using Min Heap
Program Description: Takes the inputs {arr, arr size, k} and finds the Kth
smallest element in the given array using a Min Heap
Program Creation Time: Aug 13th 04:40
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<queue>
#include<vector>
#include<functional>
using namespace std;

int kthSmallest(int arr[], int n, int k);

int main(void) {
    cout << "===== " << endl;
    cout << "FINDING KTH SMALLEST ELEMENT USING MIN HEAP" << endl;
    cout << "===== " << endl;

    int n = 0;
```

```
cout << "Enter Size of array to be Created: ";
cin >> n;
```

```
if (n <= 0) {
    cout << "Invalid array size!!" << endl;
    return 0;
}
```

```
int arr[n];
```

```
for (int i = 0; i < n; i++) {
    cout << "Enter Element " << i << " in array: ";
    cin >> arr[i];
}
```

```
cout << "-----" << endl;
```

```
int k;
```

```
cout << "Enter value of K: ";
cin >> k;
```

```
if (k < 1 || k > n) {
    cout << "Invalid value of K!!" << endl;
    return 0;
}
```

```
int result = kthSmallest(arr, n, k);
```

```
cout << "The " << k << "th smallest element is: "
<< result << endl;
```

```
cout << "-----" << endl;
cout << "Time Complexity: O(n + k log n)" << endl;
cout << "Space Complexity: O(n)" << endl;
```

```
return 0;
}
```

```
int kthSmallest(int arr[], int n, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
```

```
for (int i = 0; i < n; i++)
    minHeap.push(arr[i]);
```

```
for (int i = 1; i < k; i++)
    minHeap.pop();
```

```
return minHeap.top();
}
```

Complexity

Time Complexity: $O(n + k \log n)$

Space Complexity: $O(n)$

3. Binary Search Tree

Insertion and Deletion

Problem

Create a Binary Search Tree and perform insertion and deletion operations.

Method

A Binary Search Tree is constructed by inserting elements according to the BST property:

- Elements smaller than the current node are placed in the left subtree.
- Elements greater than the current node are placed in the right subtree.

Deletion is performed according to the structure of the node being removed.

Output:

```
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummatiomMInKandBST/Source-Code/" && g++ 3_BST.cpp -o 3_BST && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 2_SummatiomMInKandBST/Source-Code/"3_BST
=====
      BINARY SEARCH TREE
    INSERTION AND DELETION OPERATIONS
=====
Enter Number of Elements: 5
-----
Enter Elements to be Inserted:
Enter Element 1: 1
Enter Element 2: 2
Enter Element 3: 7
Enter Element 4: 3
Enter Element 5: 9
-----
BST After Insertion:
Inorder Traversal: 1 2 3 7 9
-----
Enter Element to be Deleted: 2
BST After Deletion:
Inorder Traversal: 1 3 7 9
-----
Insertion Time Complexity: O(log n) Average, O(n) Worst Case
Deletion Time Complexity: O(log n) Average, O(n) Worst Case
Space Complexity: O(n)
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 2/Source-Code
```

Code:

```
/*  
Program Name: Binary Search Tree Implementation  
Program Description: Takes the inputs {number of elements, elements} and  
creates a Binary Search Tree. Performs insertion and  
deletion operations on the Binary Search Tree  
Program Creation Time: Aug 14th 01:40  
Program Creator: Ashwath Raj | AP25110010712  
*/
```

```
#include<iostream>  
using namespace std;
```

```
class BST {  
    struct Node {  
        int val;  
        Node* left;  
        Node* right;
```

```
    Node(int value) {  
        val = value;  
        left = nullptr;  
        right = nullptr;  
    }  
};
```

```
Node* root;
```

```
Node* insert(Node* root, int val);  
Node* deleteNode(Node* root, int val);  
Node* minValueNode(Node* root);  
void inorder(Node* root);
```

```
public:  
    BST();
```

```
    void insert(int val);  
    void deleteElement(int val);  
    void display();  
};
```

```
BST::BST() {  
    root = nullptr;  
}
```

```
BST::Node* BST::insert(Node* root, int val) {  
    if (root == nullptr)  
        return new Node(val);
```

```
    if (val < root->val)
```

```
root->left = insert(root->left, val);  
else if (val > root->val)  
root->right = insert(root->right, val);  
else  
cout << "Duplicate element not allowed!!!" << endl;
```

```
return root;  
}
```

```
void BST::insert(int val) {  
root = insert(root, val);  
}
```

```
BST::Node* BST::minValueNode(Node* root) {  
Node* temp = root;
```

```
while (temp->left != nullptr)  
temp = temp->left;
```

```
return temp;  
}
```

```
BST::Node* BST::deleteNode(Node* root, int val) {  
if (root == nullptr)  
return root;
```

```
if (val < root->val)  
root->left = deleteNode(root->left, val);
```

```
else if (val > root->val)  
root->right = deleteNode(root->right, val);
```

```
else {  
if (root->left == nullptr) {  
Node* temp = root->right;  
delete root;  
return temp;  
}
```

```
if (root->right == nullptr) {  
Node* temp = root->left;  
delete root;  
return temp;  
}
```

```
Node* temp = minValueNode(root->right);  
root->val = temp->val;  
root->right = deleteNode(root->right, temp->val);  
}
```

```
return root;
}
```

```
void BST::deleteElement(int val) {
if (root == nullptr) {
cout << "BST is empty!!" << endl;
return;
}
```

```
root = deleteNode(root, val);
}
```

```
void BST::inorder(Node* root) {
if (root == nullptr)
return;
```

```
inorder(root->left);
cout << root->val << " ";
inorder(root->right);
}
```

```
void BST::display() {
if (root == nullptr) {
cout << "BST is empty!!" << endl;
return;
}
```

```
cout << "Inorder Traversal: ";
inorder(root);
cout << endl;
}
```

```
int main(void) {
cout << "===== " << endl;
cout << " BINARY SEARCH TREE" << endl;
cout << " INSERTION AND DELETION OPERATIONS" << endl;
cout << "===== " << endl;
```

```
BST tree;
```

```
int n;
cout << "Enter Number of Elements: ";
cin >> n;
```

```
if (n <= 0) {
cout << "Invalid Number of Elements!!" << endl;
return 0;
}
```

```
cout << "-----" << endl;
```

```

cout << "Enter Elements to be Inserted:" << endl;

for (int i = 0; i < n; i++) {
    int val;
    cout << "Enter Element " << i + 1 << ": ";
    cin >> val;

    tree.insert(val);
}

cout << "-----" << endl;
cout << "BST After Insertion:" << endl;
tree.display();

cout << "-----" << endl;

int val;
cout << "Enter Element to be Deleted: ";
cin >> val;

tree.deleteElement(val);

cout << "BST After Deletion:" << endl;
tree.display();

cout << "-----" << endl;
cout << "Insertion Time Complexity: O(log n) Average, O(n) Worst Case" << endl;
cout << "Deletion Time Complexity: O(log n) Average, O(n) Worst Case" << endl;
cout << "Space Complexity: O(n)" << endl;

return 0;
}

```

Complexity

Insertion: $O(\log n)$ average, $O(n)$ worst case

Deletion: $O(\log n)$ average, $O(n)$ worst case

Space Complexity: $O(n)$